Palm OS Programming Recipes

Copying a Bitmap from an Offscreen Window

Other recipes described how to draw a bitmap family member using `WinDrawBitmap` and `WinPaintBitmap`. The `WinDrawBitmap` and `WinPaintBitmap` functions check the handheld's screen depth, choose the correct bitmap family member dependent on the screen depth, and create a draw window for your application. If your application already knows the handheld's screen depth, then you can improve performance by copying the exact image you want from an offscreen window. For most applications with only light use of graphics, the performance improvement will be unnoticeable. So use this method only when performance or direct access to a bitmap are important.

1. Use Constructor for Palm OS to open the form that will contain the bitmap.

2. In the Project Window, create a new Bitmap Family.

3. Open the Bitmap Family resource and set the values for the properties:

    - Note that every member of the Bitmap Family must have the same Width and Height.
    - You can set one of the colors to be transparent when the bitmap is drawn to the screen.
    - You can add bitmaps of different depths to the bitmap family by selecting **Edit > New Family Member** (also by using Ctrl-K on Windows or Cmd-K on Macintosh). You can use up to five bitmap depths in one bitmap family, but bitmap images must be stored in increasing order of bit depth, and the bit depths must be unique. For backward compatibility, you should always have a 1-bit black and white bitmap as the first bitmap in the bitmap family.

      Note that although individual bitmap members appear to have their own resource IDs in Constructor, you can only use the resource ID of the bitmap family. If you are checking the screen depth and choosing the bitmap yourself, create bitmap resources that are not part of a family.

4. Set up an offscreen window containing the bitmap.

   The time and work for copying from an offscreen window is in set up and allocation. To improve the performance of drawing the bitmap, place this code in your `AppStart` function.

   The `BmpGlueGetDiminsions` API was added in the OS 4.0 SDK. For backwards compatibility, use the PalmOSGlue version. Add `PalmOSGlue.lib` to your project and include the `BmpGlue.h` header file.

   If you plan on changing the screen depth, you need to do it before you create your offscreen window because `WinCreateOffscreenWindow` creates the offscreen window in the depth of the screen.

```
#include "BmpGlue.h"

// given that there is a global declared
// as WinHandle global_bitmapWinH;
UInt16 error;
Coord bitmapWidth, bitmapHeight;
UInt16 bitmapRowBytes;

//Get a pointer to the bitmap
MemHandle bitmapH = DmGetResource(bitmapRsc, TestBitmapFamily);

ErrFatalDisplayIf(!bitmapH, "Could Not Get Bitmap Resource");
BitmapType* bitmapP = (BitmapType*) MemHandleLock(bitmapH);

//use glue code from OS 4.0 SDK to get width and height of bitmap
//(see BmpGlue.h)
BmpGlueGetDimensions (bitmapP, &bitmapWidth, &bitmapHeight,
        &bitmapRowBytes);

//create an offscreen window for the bitmap
global_bitmapWinH = WinCreateOffscreenWindow(bitmapWidth,
bitmapHeight, screenFormat, &error);
ErrFatalDisplayIf(error, "Could Not Create Offscreen Window");

//draw the bitmap to the offscreen window
//note that the offscreen window created has the same
//depth as the screen and WinDrawBitmap will draw the
//correct bitmap member from the family

WinHandle screenH = WinSetDrawWindow(global_bitmapWinH);
WinDrawBitmap(bitmapP, 0, 0);

//return the draw window to the screen
WinSetDrawWindow(screenH);

//you no longer need the bitmapP
MemHandleUnlock(bitmapH);
DmReleaseResource(bitmapH);
```

5. Copy your image to the screen.

   After creating your offscreen window, you have your image in a draw window waiting to be copied to the screen. You can safely modify it directly using `WinGetBitmap` to get the window's bitmap (remember the image here is a copy of the original, not the bitmap stored as a resource). Then you can get the bits with `BmpGetBits`. Once you are ready to copy it to the screen, you can do it very quickly with `WinCopyRectangle`. `WinCopyRectangle` can use the various copy modes defined under `WinDrawOperation` in *Palm OS Programmer's API Reference*. The code to copy your image to the screen is as follows:

```
RectangleType bounds;
```

```
//if you do not already have it, get the handle to the screen
WinHandle screenH = WinGetDrawWindow();

WinGetBounds(global_bitmapWinH, &bounds);

//given that drawToX, and drawToY are the top left coords of
//where you want to draw to...
//Also, you can substitute other copy modes for winPaint
WinCopyRectangle(global_bitmapWinH, screenH, &bounds,
        drawToX, drawToY, winPaint);
```

You need to free the offscreen window before the application exits, for example, in the `AppStop` function:

```
WinDeleteWindow(global_bitmapWinH, false);
```

In the case where the bitmap resides in the dynamic heap (one that was created with `BmpCreate` or completely copied over from the storage heap), you may want to use `WinCreateBitmapWindow` in place of `WinCreateOffscreenWindow`. For more information, see *Palm OS Programmer's API Reference*.

The following related recipes describe alternative methods for drawing bitmaps to the screen:

- Adding a Bitmap Family to a Form
- Drawing a Bitmap Family Member
- Using `WinPaintBitmap` Instead of `WinDrawBitmap`